

MAINTENANCE MANAGEMENT SYSTEM

Project Handover Document

Document Type	Project Handover Document
Intern Developer	Mr. Jaice Jommy
Mentor	Mr. Srinivasan Devaraj
Application	Maintenance Management System (MMS)
Status	Completed

Table of Contents

Table of Contents	2
1. System Overview	3
2. Handover Details	3
3. System Prerequisites & Requirements	3
Hardware Requirements	3
Software Prerequisites	3
4. Tech Stack & Dependencies	3
Backend Tech Stack	3
Key Python Dependencies (requirements.txt)	3
Frontend Tech Stack	4
5. System Architecture & Components	4
Backend Architecture	4
Frontend Architecture	4
6. Database Schema & Storage	5
Core Tables	5
File Storage	5
7. Security & Roles	5
User Roles	5
Security Measures	5
8. Setup & Execution Guide	5
9. Offline Deployment & Installer	6

1. System Overview

The Maintenance Management System (MMS) is a plant intranet CRUD (Create, Read, Update, Delete) application designed for capturing, tracking, and managing manual maintenance records. It provides a secure, role-based platform for engineers, operators, and administrators to digitize equipment maintenance workflows.

2. Handover Details

As part of this handover, the system has been prepared with the following initial states:

- Database State: The database is completely clean and empty of any maintenance or equipment data.
- Default Credentials: For initial access, use Username: admin and Password: AdminPass123!

3. System Prerequisites & Requirements

Hardware Requirements

- Host Server: Any standard x86_64 or ARM64 server/PC capable of running Python 3.11+.
- Network: Local Area Network (LAN) accessibility for intranet users.
- Client: Any modern web browser (Chrome, Edge, Firefox, Safari).

Software Prerequisites

- Operating System: Windows, Linux, or macOS.
- Runtime: Python 3.11 or later.
- Package Manager: pip (bundled with Python).
- Environment: Virtual Environment (venv) is highly recommended for dependency isolation.

4. Tech Stack & Dependencies

The application relies on a modern, zero-build-step architecture.

Backend Tech Stack

- Language: Python 3.11+
- Framework: FastAPI (High-performance web framework for building APIs).
- ASGI Server: Uvicorn (Handles HTTP requests asynchronously).
- Database: SQLite3 (File-based database, no external DB server required).
- Authentication: JWT (JSON Web Tokens) with Bcrypt password hashing.

Key Python Dependencies (requirements.txt)

- fastapi==0.111.0: Core REST API framework.
- uvicorn[standard]==0.29.0: ASGI server for running the app.

- passlib[bcrypt]==1.7.4 & bcrypt==4.0.1: Password hashing and verification.
- PyJWT==2.10.1: JWT creation and validation for authentication.
- python-multipart==0.0.9: Handling form data and file uploads.
- python-dotenv==1.0.1: Loading environment variables from .env.
- reportlab==5.0.0: PDF generation for exporting maintenance records.
- pydantic==2.13.4: Data validation and settings management.

Frontend Tech Stack

- Language: HTML5, CSS3, Vanilla JavaScript (ES6+).
- Styling: Custom CSS and Bootstrap (via bundled local files).
- Architecture: Single Page Application (SPA) feel, using asynchronous fetch() API calls to communicate with the backend.
- Build Tools: None. The frontend consists of pure static files served directly by the backend.

5. System Architecture & Components

The system follows a classic Client-Server Architecture, using FastAPI to provide a RESTful JSON API and serve static HTML/JS/CSS assets for the frontend interface.

Backend Architecture

The backend (located in backend/) follows a layered structure separating routing, business logic/data access, and data validation:

- **Core Configuration & Entry (backend/main.py):** Defines the FastAPI application instance, manages application lifecycle, configures CORS middleware, registers a global exception handler, and mounts the frontend/ directory.
- **Data Access Layer (backend/database.py):** Manages the SQLite database connection, utilizing the sqlite3.Row factory. Enables Write-Ahead Logging (WAL) and foreign keys. Exposes parameterized helper functions (fetch_one, fetch_all, execute). Responsible for auto-generating tables.
- **Authentication & Authorization (backend/auth.py):** Handles password hashing via Passlib (Bcrypt). Generates and decodes JWT tokens. Provides FastAPI dependency injections (get_current_user, require_role).
- **API Routing Controllers (backend/api/):** Includes record_routes.py, equipment_routes.py, user_routes.py, csv_routes.py & export_routes.py. Handles complex CRUD logic, file uploads, role management, and bulk operations.
- **Data Validation Models (backend/models/):** Pydantic models used to parse, sanitize, and validate JSON payloads from incoming HTTP requests and format outbound HTTP responses.

Frontend Architecture

The frontend operates as a Single Page Application (SPA) natively using Vanilla JavaScript and standard HTML5 without a Node.js build process.

- **Pages (.html):** Logical pages like dashboard.html, form.html, and record-detail.html. Includes Bootstrap for responsive layout grids.
- **Client-Side Logic (frontend/js/):** api.js (central fetch wrapper), auth.js (manages login process/JWT), ui.js (dynamic DOM manipulation), app.js (connects HTML elements to logic), and validators.js (client-side form validation).

6. Database Schema & Storage

The database is powered by SQLite (backend/storage/maintenance.db). It utilizes Write-Ahead Logging (WAL) for better concurrency.

Core Tables

- **users:** Stores user credentials, hashed passwords, roles, and active status.
- **maintenance_records:** Stores core maintenance data (type, equipment path, responsible person, timestamps, audit trails).
- **record_csv_data:** Stores bulk CSV data associated with specific maintenance records.
- **record_attachments:** Tracks metadata for uploaded files/attachments linked to records.
- **equipment_hierarchy:** Stores the plant equipment tree (Enterprise > Site > Area > Work Center > Work Unit > Equipment ID).

File Storage

- Database File: backend/storage/maintenance.db
- Attachments: Uploaded files are physically stored in the backend/storage/attachments/ directory.

Backup Strategy: Regular backups must include both the maintenance.db file and the entire attachments/ folder to prevent data loss.

7. Security & Roles

The system implements Role-Based Access Control (RBAC) to restrict functionalities based on user privileges.

User Roles

- Administrator: Full system access. Can create/edit/delete all records, manage the equipment hierarchy, and manage user accounts (create users, reset passwords).
- Engineer / Operator: Standard users who can create new maintenance records and edit only their own records.
- Viewer: Read-only access. Can view records and dashboards but cannot make modifications.

Security Measures

- Passwords are never stored in plaintext (hashed via Bcrypt).
- APIs are protected using JWT Bearer tokens.
- No traceback or sensitive internal errors are exposed to the client (Global Exception Handler in main.py).

8. Setup & Execution Guide

1. Clone the Repository and navigate to the project root.
GitHub repository URL - https://github.com/jaicejommy/ykg_maintenance_app.git

2. Setup Virtual Environment: `python -m venv venv`
3. Activate Virtual Environment: `.\venv\Scripts\Activate.ps1` (Windows) or `source venv/bin/activate` (Linux/Mac)
4. Install Dependencies: `pip install -r backend/requirements.txt`
5. Environment Configuration: Copy `.env.example` to `.env` and set secure values.
6. Run the Application: `uvicorn backend.main:app --host 0.0.0.0 --port 8000`
7. Access the App: Navigate to `http://<server-ip>:8000` in a web browser. API documentation is available at `http://<server-ip>:8000/api/docs`

9. Offline Deployment & Installer

The application is packaged with an offline deployment system (located in `installer/`) meant for isolated plant intranet environments without internet access.

- `download_wheels.bat`: A script run on an internet-connected machine to download all Python `.whl` dependencies offline into a `wheels/` directory.
- `setup.iss`: An Inno Setup script that compiles a `MMS_Setup_v1.0.exe` installer. It silently installs Python 3.11, creates the virtual environment, installs the offline pip wheels, and configures shortcuts.
- `USER_MANUAL.txt`: End-user instructions for operating the compiled installer.